

100 Python Interview Questions with KISS Answers, Approach, and Logic Building

How to Use This Guide

For every coding question follow this interview flow:

1. Understand input and output
2. Think brute force first
3. Optimize later
4. Explain time complexity
5. Write clean code
6. Test with sample input

Interview Formula:

- Requirement → Logic → Steps → Code → Edge Cases
-

SECTION 1: BASIC PYTHON QUESTIONS

1. Reverse a String

Question

Reverse a string without using slicing.

Approach

- Take empty string
- Loop from last index to first
- Append characters

Logic

If string is "abc"

- start from c
- then b
- then a

KISS Answer

"I will iterate from end to start and build a new reversed string."

Code

```
s = "python"
rev = ""

for i in range(len(s)-1, -1, -1):
    rev += s[i]

print(rev)
```

Time Complexity

O(n)

2. Check Palindrome

Question

Check whether a string is palindrome.

Approach

- Reverse string
- Compare with original

Logic

madam == madam

KISS Answer

"Palindrome means same from front and back."

Code

```
s = "madam"

if s == s[::-1]:
    print("Palindrome")
```

```
else:  
    print("Not Palindrome")
```

3. Find Largest Number in List

Approach

- Assume first number is largest
- Compare all elements

Code

```
nums = [4, 2, 9, 1]  
largest = nums[0]  
  
for n in nums:  
    if n > largest:  
        largest = n  
  
print(largest)
```

Interview Explanation

"I compare every element with current maximum and update it if bigger value comes."

4. Find Smallest Number in List

Code

```
nums = [4, 2, 9, 1]  
smallest = nums[0]  
  
for n in nums:  
    if n < smallest:  
        smallest = n  
  
print(smallest)
```

5. Find Sum of List

Logic

- Start sum from 0
- Add every number

Code

```
nums = [1, 2, 3, 4]
total = 0

for n in nums:
    total += n

print(total)
```

6. Count Vowels in String

Approach

- Loop every character
- Check if vowel

Code

```
s = "education"
count = 0

for ch in s:
    if ch.lower() in 'aeiou':
        count += 1

print(count)
```

7. Swap Two Variables

Code

```
a = 10
b = 20
```

```
a, b = b, a  
  
print(a, b)
```

KISS Explanation

"Python supports tuple unpacking so extra variable is not needed."

8. Find Factorial

Logic

$5! = 5 \times 4 \times 3 \times 2 \times 1$

Code

```
n = 5  
fact = 1  
  
for i in range(1, n+1):  
    fact *= i  
  
print(fact)
```

9. Fibonacci Series

Approach

- Store previous two numbers
- Generate next using sum

Code

```
n = 7  
a, b = 0, 1  
  
for _ in range(n):  
    print(a)  
    a, b = b, a+b
```

10. Prime Number Check

Logic

Prime divisible only by 1 and itself.

Code

```
n = 11
is_prime = True

for i in range(2, n):
    if n % i == 0:
        is_prime = False
        break

print(is_prime)
```

SECTION 2: STRING QUESTIONS

11. First Non-Repeating Character

Approach

- Count characters
- Return first count == 1

Code

```
s = "aabbcdde"

for ch in s:
    if s.count(ch) == 1:
        print(ch)
        break
```

12. Remove Duplicates from String

Code

```
s = "aabbcc"
result = ""

for ch in s:
    if ch not in result:
        result += ch

print(result)
```

13. Count Character Frequency

Code

```
s = "apple"
freq = {}

for ch in s:
    freq[ch] = freq.get(ch, 0) + 1

print(freq)
```

14. Check Anagram

Logic

Sorted strings should be equal.

Code

```
s1 = "listen"
s2 = "silent"

print(sorted(s1) == sorted(s2))
```

15. Capitalize First Letter

Code

```
s = "hello world"
print(s.title())
```

16. Count Words in Sentence

Code

```
s = "python is easy"
print(len(s.split()))
```

17. Find Duplicate Characters

Code

```
s = "programming"
seen = set()

for ch in s:
    if s.count(ch) > 1:
        seen.add(ch)

print(seen)
```

18. Replace Spaces with Hyphen

Code

```
s = "hello world"
print(s.replace(" ", "-"))
```

19. Reverse Words in Sentence

Code

```
s = "I love python"
words = s.split()

print(" ".join(words[::-1]))
```

20. Check String Contains Digits

Code

```
s = "abc123"

for ch in s:
    if ch.isdigit():
        print("Contains digit")
        break
```

SECTION 3: LIST QUESTIONS

21. Remove Duplicates from List

Code

```
nums = [1,2,2,3,4,4]
result = []

for n in nums:
    if n not in result:
        result.append(n)

print(result)
```

22. Find Second Largest Number

Approach

- Sort list
- Pick second last

Code

```
nums = [1, 5, 9, 3]
nums.sort()

print(nums[-2])
```

23. Merge Two Lists

Code

```
l1 = [1,2]
l2 = [3,4]

print(l1 + l2)
```

24. Find Common Elements in Two Lists

Code

```
l1 = [1,2,3]
l2 = [2,3,4]

result = []

for n in l1:
    if n in l2:
        result.append(n)

print(result)
```

25. Rotate List

Code

```
nums = [1,2,3,4,5]
k = 2

print(nums[-k:] + nums[:-k])
```

26. Find Missing Number

Logic

Expected sum - actual sum

Code

```
nums = [1,2,3,5]
n = 5

expected = n * (n+1) // 2
actual = sum(nums)

print(expected - actual)
```

27. Sort Without sort()

Code

```
nums = [4,2,1,3]

for i in range(len(nums)):
    for j in range(i+1, len(nums)):
        if nums[i] > nums[j]:
            nums[i], nums[j] = nums[j], nums[i]

print(nums)
```

28. Find Maximum Difference

Code

```
nums = [2,7,1,9]

print(max(nums) - min(nums))
```

29. Flatten Nested List

Code

```
nums = [[1,2],[3,4]]
result = []

for sub in nums:
    for item in sub:
        result.append(item)

print(result)
```

30. Check List is Sorted

Code

```
nums = [1,2,3,4]

print(nums == sorted(nums))
```

SECTION 4: DICTIONARY QUESTIONS

31. Merge Two Dictionaries

```
d1 = {'a':1}
d2 = {'b':2}
```

```
print(**d1, **d2)
```

32. Sort Dictionary by Value

```
d = {'a':3, 'b':1, 'c':2}

print(sorted(d.items(), key=lambda x: x[1]))
```

33. Invert Dictionary

```
d = {'a':1, 'b':2}

result = {v:k for k,v in d.items()}
print(result)
```

34. Find Key with Maximum Value

```
d = {'a':10, 'b':20}

print(max(d, key=d.get))
```

35. Count Word Frequency

```
s = "python is easy and python is powerful"
words = s.split()
result = {}

for w in words:
    result[w] = result.get(w, 0) + 1

print(result)
```

SECTION 5: PATTERN QUESTIONS

36. Star Pyramid

```
n = 5

for i in range(1, n+1):
    print('*' * i)
```

37. Reverse Pyramid

```
n = 5

for i in range(n, 0, -1):
    print('*' * i)
```

38. Number Triangle

```
n = 5

for i in range(1, n+1):
    print(str(i) * i)
```

39. Floyd Triangle

```
n = 5
num = 1

for i in range(1, n+1):
    for j in range(i):
        print(num, end=' ')
        num += 1
    print()
```

40. Right Aligned Triangle

```
n = 5

for i in range(1, n+1):
    print(' '*(n-i) + '*'*i)
```

SECTION 6: SEARCHING AND SORTING

41. Linear Search

Logic

Check each element one by one.

```
nums = [2,4,6,8]
target = 6

for i in nums:
    if i == target:
        print('Found')
```

42. Binary Search

Logic

Works on sorted array.

```
nums = [1,2,3,4,5]
target = 4

low = 0
high = len(nums)-1

while low <= high:
    mid = (low + high)//2

    if nums[mid] == target:
        print('Found')
```

```
        break
    elif nums[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
```

43. Bubble Sort

```
nums = [4,2,1,3]

for i in range(len(nums)):
    for j in range(len(nums)-1):
        if nums[j] > nums[j+1]:
            nums[j], nums[j+1] = nums[j+1], nums[j]

print(nums)
```

44. Selection Sort

```
nums = [4,2,1,3]

for i in range(len(nums)):
    min_index = i

    for j in range(i+1, len(nums)):
        if nums[j] < nums[min_index]:
            min_index = j

    nums[i], nums[min_index] = nums[min_index], nums[i]

print(nums)
```

45. Insertion Sort

```
nums = [4,2,1,3]

for i in range(1, len(nums)):
    key = nums[i]
```



```
j = i - 1

while j >= 0 and nums[j] > key:
    nums[j+1] = nums[j]
    j -= 1

nums[j+1] = key

print(nums)
```

SECTION 7: FUNCTIONS AND OOPS

46. Difference Between List and Tuple

KISS Answer

- List is mutable
- Tuple is immutable
- Tuple faster than list

47. What is *args?

Answer

Used to pass multiple positional arguments.

```
def add(*args):
    return sum(args)
```

48. What is **kwargs?

Answer

Used to pass keyword arguments.

```
def show(**kwargs):
    print(kwargs)
```

49. What is Inheritance?

KISS Answer

Child class can use parent properties.

```
class A:
    def show(self):
        print('Parent')

class B(A):
    pass
```

50. What is Polymorphism?

KISS Answer

Same method behaves differently.

51. What is Encapsulation?

KISS Answer

Wrapping data and methods together.

52. What is Abstraction?

KISS Answer

Showing only important details and hiding implementation.

53. Difference Between @staticmethod and @classmethod

Answer

- staticmethod → no class reference
- classmethod → uses cls

54. Constructor in Python

```
class Test:
    def __init__(self):
        print('Constructor')
```

55. What is super()?

Answer

Used to access parent class methods.

SECTION 8: FILE HANDLING

56. Read File

```
with open('test.txt', 'r') as file:
    print(file.read())
```

57. Write File

```
with open('test.txt', 'w') as file:
    file.write('Hello')
```

58. Append File

```
with open('test.txt', 'a') as file:
    file.write('World')
```

59. Count Lines in File

```
with open('test.txt') as file:  
    print(len(file.readlines()))
```

60. Copy File

```
with open('a.txt') as f1:  
    data = f1.read()  
  
with open('b.txt', 'w') as f2:  
    f2.write(data)
```

SECTION 9: EXCEPTION HANDLING

61. Try Except Example

```
try:  
    print(10/0)  
except ZeroDivisionError:  
    print('Cannot divide by zero')
```

62. Finally Block

Answer

Finally always executes.

63. Raise Exception

```
age = -1  
  
if age < 0:  
    raise ValueError('Invalid age')
```

64. Custom Exception

```
class MyError(Exception):  
    pass
```

65. Difference Between Error and Exception

Answer

- Exception can be handled
- Error mostly system related

SECTION 10: ADVANCED PYTHON

66. What is Lambda Function?

```
square = lambda x: x*x  
print(square(5))
```

67. Map Function

```
nums = [1,2,3]  
  
print(list(map(lambda x:x*2, nums)))
```

68. Filter Function

```
nums = [1,2,3,4]  
  
print(list(filter(lambda x:x%2==0, nums)))
```

69. Reduce Function

```
from functools import reduce

nums = [1,2,3,4]

print(reduce(lambda x,y:x+y, nums))
```

70. List Comprehension

```
nums = [x*x for x in range(5)]
print(nums)
```

71. Generator Function

```
def gen():
    yield 1
    yield 2
```

72. Difference Between List and Generator

Answer

- List stores all values in memory
 - Generator generates values one by one
-

73. Decorator Example

```
def outer(func):
    def inner():
        print('Before')
        func()
    return inner
```

74. Iterator vs Iterable

Answer

- Iterable can be looped
 - Iterator gives next value using next()
-

75. Deep Copy vs Shallow Copy

Answer

- Shallow copy references nested objects
 - Deep copy creates complete new object
-

SECTION 11: PYTHON + DJANGO INTERVIEW

76. What is Django?

KISS Answer

Django is high-level Python web framework for rapid development.

77. What is MVT Architecture?

Answer

- Model → Database
 - View → Business logic
 - Template → UI
-

78. Difference Between GET and POST

Answer

- GET retrieves data
 - POST sends secure data
-

79. What is ORM?

KISS Answer

ORM converts Python queries into SQL queries.

80. What is Migration?

Answer

Migration updates database schema.

81. What is Django REST Framework?

Answer

Used for creating APIs quickly.

82. Serializer in DRF

Answer

Converts complex data into JSON.

83. Authentication vs Authorization

Answer

- Authentication → who are you
 - Authorization → what you can access
-

84. What is Middleware?

Answer

Middleware processes request and response globally.

85. What is CSRF?

Answer

Security protection against fake requests.

86. Difference Between select_related and prefetch_related

Answer

- select_related → ForeignKey join
 - prefetch_related → separate query for ManyToMany
-

87. What is API Pagination?

Answer

Splits large data into smaller pages.

88. What is JWT?

Answer

Token-based authentication mechanism.

89. What is Caching?

Answer

Stores frequently used data temporarily for speed.

90. Explain Add to Cart API Approach

Interview Flow

1. Authenticate user
2. Check product exists
3. Check stock quantity
4. Check cart already has product

5. Update quantity or create cart item
6. Return response

KISS Answer

"First I validate user and product. Then I check stock and whether item already exists in cart. If yes, update quantity otherwise create new cart item. Finally return updated cart response."

SECTION 12: LOGIC BUILDING QUESTIONS

91. Two Sum Problem

Logic

Find two numbers whose sum equals target.

```
nums = [2,7,11,15]
target = 9
seen = {}

for i, num in enumerate(nums):
    diff = target - num

    if diff in seen:
        print(seen[diff], i)

    seen[num] = i
```

92. Move Zeroes to End

```
nums = [0,1,0,3,12]
result = []
zeroes = []

for n in nums:
    if n == 0:
        zeroes.append(n)
    else:
        result.append(n)

print(result + zeroes)
```

93. Find Duplicate Number

```
nums = [1,2,3,2]
seen = set()

for n in nums:
    if n in seen:
        print(n)
        break
    seen.add(n)
```

94. Maximum Consecutive Ones

```
nums = [1,1,0,1,1,1]
count = 0
max_count = 0

for n in nums:
    if n == 1:
        count += 1
        max_count = max(max_count, count)
    else:
        count = 0

print(max_count)
```

95. Valid Parentheses

```
s = '()[{}]'
stack = []
map = {'(': ')', '[': ']', '{': '}'

for ch in s:
    if ch in map.values():
        stack.append(ch)
    else:
        if not stack or stack.pop() != map[ch]:
            print(False)
            break
```

```
else:  
    print(True)
```

96. Longest Word in Sentence

```
s = 'Python is very powerful'  
words = s.split()  
  
print(max(words, key=len))
```

97. Find Intersection of Arrays

```
nums1 = [1,2,2,1]  
nums2 = [2,2]  
  
print(list(set(nums1) & set(nums2)))
```

98. Find Even Numbers

```
nums = [1,2,3,4,5]  
  
for n in nums:  
    if n % 2 == 0:  
        print(n)
```

99. Check Armstrong Number

```
n = 153  
sum = 0  
temp = n  
  
while temp > 0:  
    digit = temp % 10  
    sum += digit ** 3  
    temp //= 10
```

```
print(sum == n)
```

100. Roman to Integer

```
roman = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100}
s = 'IX'
result = 0

for i in range(len(s)-1):
    if roman[s[i]] < roman[s[i+1]]:
        result -= roman[s[i]]
    else:
        result += roman[s[i]]

result += roman[s[-1]]

print(result)
```

FINAL INTERVIEW TIPS FOR PYTHON DEVELOPERS

Coding Round Strategy

Step 1: Clarify

Ask:

- Input type?
- Constraints?
- Expected output?

Step 2: Explain Brute Force

Interviewers love thought process.

Step 3: Optimize

Use:

- HashMap
- Set

- Two Pointer
- Sliding Window
- Binary Search

Step 4: Dry Run

Always test with sample input.

Step 5: Mention Complexity

Example:

- Time Complexity $\rightarrow O(n)$
- Space Complexity $\rightarrow O(1)$

Most Important Topics for Python Django Developer Interviews

1. String manipulation
2. List and dictionary logic
3. API flow explanation
4. OOPS concepts
5. Django ORM
6. Authentication
7. SQL joins
8. REST APIs
9. Optimization
10. Problem-solving approach

Golden Rule for Interviews

"Do not jump directly into code. First explain logic in simple English."

Your interview flow should look like this:

Requirement $\rightarrow$ Approach $\rightarrow$ Logic $\rightarrow$ Edge Cases $\rightarrow$ Code $\rightarrow$ Complexity

That alone makes you look like experienced engineer.